GeminiClaw: AI 기반 회사 운영 OS — 기획문서

> **작성일**: 2026년 2월 24일
> **프로젝트**: GeminiClaw (openclaw_backend + openclaw_app)
> **목표**: AI 에이전트 팀이 회사 운영을 자동화하는 시스템

---

1. 프로젝트 개요

1.1 비전

1인 창업자(CTO)가 AI 에이전트 팀을 구성하여 회사 운영 전반(법무, 회계, 마케팅, 개발 등)을 자동화하는 **"AI 회사 운영 OS"**.

1.2 대상 유저 및 확장성

- **주 타겟**: 시간과 리소스가 부족한 1인 창업자, 초기 스타트업 CTO, 소규모 비즈니스 대표
- **특징 (Dynamic Multi-Tenant)**: 특정 대상 회사(예: PT Humantric)에 종속되지 않고, 사용자가 최초에 입력한 **'회사 기본 프로필(도메인, 목표 등)'**을 바탕으로 Gemini AI가 해당 비즈니스에 최적화된 맞춤형 조직과 업무 프로세스를 런타임에 동적으로 설계합니다.

---

2. 핵심 운영 메커니즘: 동적 온보딩 및 지속 운영 (Continuous Operation)

시스템은 단순한 1회성 챗봇이 아닌, **스스로 업무 매뉴얼을 구축하고 이를 기반으로 영속적으로 동작하는 OS**로 기능합니다.

2.1 AI 자동 온보딩 파이프라인 (Setup & Initialization)

1. **사용자 입력 (Company Profiling)**: 앱스토어 런칭 준비 중인 IT 기업, 글로벌 무역 회사, 동네 카페 등 사용자가 자신의 회사 기본 정보와 현재 목표를 프롬프트로 입력합니다.
2. **조직도 자동 설계 (Org Diagramming)**: AI가 입력된 비즈니스 모델을 분석하여 필수적인 부서(예: 개발, 법무, 번역, 마케팅)와 에이전트 구성을 프론트엔드에 제안합니다.
3. **업무 매뉴얼/스킬셋 동적 생성 (Skill Documentation)**: 확정된 팀 구성에 맞춰, AI가 각 부서별 **표준 업무 지침서(SOP, Standard Operating Procedures)나 Skill 마크다운 문서**를 자동으로 생성하여 로컬 워크스페이스(예: `docs/skills/{role}_manual.md`)에 저장합니다.

- *예: 마케터 에이전트 생성 시 → `App Store 최적화(ASO) 스킬셋`, `인스타그램 일일 포스팅 규정` 등의 문서를 AI가 스스로 작성하여 저장.*

2.2 지속적 운영 시스템 (Continuous Execution Loop)

- **문서(Skill) 기반 능동적 실행 (RAG 융합)**: PM 에이전트나 각 실무 에이전트는 태스크를 할당받으면, 즉시 코드를 짜거나 글을 쓰는 대신 **이전에 자동 생성해둔 자신들의 부서별 Skill 문서를 먼저 읽고(read_file)**, 정해진 사내 규칙이나 포맷팅 가이드라인을 엄격히 준수하여 결과물을 도출합니다.
- **Self-Evolution (자가 발전)**: 회사가 성장함에 따라 사용자가 PM에게 피드백을 주면, PM 에이전트가 해당 부서의 Skill 문서를 직접 업데이트(`write_file`)하여 다음 작업부터는 새로운 규정이 적용되도록 시스템 스스로 진화합니다.

---

3. 시스템 아키텍처

3.1 전체 프로세스

```
사용자정보 입력 → [AI 온보딩: 조직도/Skill 문서 자동 생성] → (프로젝트 워크스페이스 세팅)
                                    ↓
CTO일상지시 → PM Agent→[Skill 문서참조] → [부서별실무 Agent]→Reviewer 검수→ 결과
                                    ↑                                    ↓
                   (규정 변경 시 Skill 매뉴얼 자동 갱신)              순차 실행 (Dispatcher)
```

3.2 기술 스택

구분	기술
Backend	Python + FastAPI + LangGraph + Gemini 2.5 Flash
Frontend	Flutter Web (Dart)
통신	WebSocket (실시간 스트리밍)
AI 모델	Google Gemini 2.5 Flash Lite
보안	HITL (Human-in-the-Loop) — 위험 명령어 CTO 승인

3.3 핵심 파일 구조

```
openclaw_backend/
├── agents/
│   ├── agent_config.py      # 9개 역할 정의 + 5개 팀 프리셋 + COMPANY_CONTEXT
│   ├── agent_factory.py     # 동적 노드 생성 + Reviewer status 파싱
│   ├── company_setup.py     # AI 조직도 생성기
│   ├── graph.py             # LangGraph 워크플로우 (SqliteSaver 상태 유지)
│   ├── pm_agent.py          # PM 에이전트 (태스크 분배)
│   ├── pi_agent.py          # 베이스 개인 비서 (1:1 채팅)
│   └── state.py             # AgentState
├── api/
│   ├── websockets.py        # WebSocket API (org_chart, task, HITL, pi_chat)
│   └── rest.py              # REST API (프로젝트/팀 프리셋 CRUD)
├── core/
│   ├── config.py            # 환경 변수 및 설정 (샌드박스 설정 포함)
│   ├── db.py                # 프로젝트 및 팀 메타데이터 보관 (SQLite)
│   └── memory.py            # Pi 세션 영구 저장 어댑터
├── sandbox/
│   ├── Dockerfile.sandbox   # Worker용 격리 컨테이너 이미지
│   └── sandbox_manager.py   # 컨테이너 생명주기 관리 및 exec 모듈
├── tools/
│   ├── file_tools.py        # read_file, write_file
│   ├── system_tools.py      # macOS 제어 (알림, 클립보드)
│   └── shell_tools.py       # execute_shell_command (DOCKER 샌드박스 및 HITL 통
합)
└── main.py                  # FastAPI 라우터 모음

openclaw_app/
└── lib/
    └── main.dart            # Dual-Mode UI (Pi Assistant / Team Orchestrator)
```

---

3. Agent 역할 (9개)

Agent	역할	도구 권한
PM	CTO 지시 분석 → 팀원에게 하위 태스크 분배	없음 (순수 라우팅)
Developer	코드 작성/수정/빌드	read_file, write_file, shell
Reviewer	품질 검수 (APPROVED/REJECTED)	read_file
Legal	이용약관, 개인정보처리방침, PSE 등록	read_file, write_file
Accountant	SPT 세무신고, VAT, 재무제표	read_file, write_file
Admin	PMA 유지관리, LKPM 보고, NIB 갱신	read_file, write_file

Agent	역할	도구 권한
Marketer	ASO, 소셜미디어, 커뮤니티 시딩	read_file, write_file
CS	FAQ, 사용자 문의 템플릿, 신고 처리	read_file, write_file
HR	채용, BPJS, 근로계약서	read_file, write_file

---

4. Mozzy 앱 분석 방안

4.1 현재 에이전트가 접근 가능한 방법

방법	가능 여부	설명
로컬 프로젝트 폴더 (`C:\bling\bling_app`)	✅ 가능	`read_file` 도구로 직접 읽기 가능
GitHub repo (`github.com/jbak2588/bling`)	⚠️ 간접 가능	`execute_shell_command` 로 `git clone` 또는 `curl` 가능
GitHub API	❌ 직접 불가	별도 도구 추가 필요 (향후)

4.2 권장 방식: 로컬 폴더 참조

에이전트의 `read_file` 도구에 `C:\bling\bling_app` 경로를 직접 지정하면
Dart 소스코드를 읽고 분석할 수 있습니다.

예시: Legal Agent가 개인정보처리방침을 작성할 때,

```
read_file("C:\bling\bling_app\lib\models\user_model.dart")
```

→ 사용자 데이터 구조를 확인 → 어떤 개인정보를 수집하는지 파악 → 문서에 반영

4.3 향후 확장: GitHub 연동 도구

```
# 추가 예정 도구
TOOL_REGISTRY = {
    "read_file": read_file,
    "write_file": write_file,
    "execute_shell_command": execute_shell_command,
    "github_read_file": github_read_file,      # GitHub API로 파일 읽기
    "github_list_files": github_list_files,     # 레포 파일 목록
```

```
    "github_search_code": github_search_code,   # 코드 검색
}
```

---

5. 개발 로드맵 (진행 완료 및 계획)

Phase 0: Base Personal Assistant (✅ 완료)

항목	상태	내용
Pi Agent (개인비서)	✅	PM을 거치지 않는 직접 1:1 채팅 에이전트 구축
로컬 시스템 툴 연동	✅	macOS 알림 발송, 클립보드 읽기/쓰기 도구 추가
세션 메모리 보관	✅	SQLite를 활용한 Pi 에이전트의 이전 대화 기억

Phase 1: Core Agent Engine & PoC (✅ 완료)

항목	상태	내용
AI 조직도 생성	✅	LLM이 회사 정보 기반 부서 추천
9개 에이전트 역할	✅	PM, Dev, Reviewer, Legal 등 역할 및 권한 분리
순차 실행 (Dispatcher)	✅	PM → Agent1 → Reviewer → Agent2 → ... → END
COMPANY_CONTEXT 주입	✅	모든 에이전트에 PT Humantric 정보 자동 제공
Knowledge 참조 시스템	✅	로컬 폴더(.md, .txt) 직접 참조 및 분석

Phase 2: Live Observability & API (✅ 완료)

항목	상태	내용
WebSocket 오케스트레이션	✅	프론트엔드로 실시간 로그 및 생각 스트리밍 연동
공유 메모리 보관 (State DB)	✅	LangGraph `SqliteSaver` 연동으로 멀티 에이전트 컨텍스트 보존
프로젝트 & 팀 REST API	✅	팀 프리셋 저장, 신규 프로젝트 워크스페이스 생성 API 오픈

Phase 3: Dedicated UI/UX Dashboard (✅ 완료)

항목	상태	내용
Dual-mode UI (Flutter)	✅	좌측 네비게이터를 통한 Pi Chat / Team OS 분리 적용
칸반 보드 뷰 (Kanban)	✅	TODO → IN_PROGRESS → REVIEW → DONE 자동 이동 시각화
HITL 팝업 연동	✅	위험 명령어 감지 시 앱 화면에서 승인/거절 UI 처리

Phase 4: Security Rules & Sandbox Integration (✅ 완료)

항목	상태	내용
Docker 샌드박스 메커니즘	✅	Worker 에이전트의 Shell 명령어는 호스트와 격리된 컨테이너 내부에서만 병렬 실행
Trust Boundary 설정	✅	Pi Agent는 Host에서 네이티브 도구를, 일반 에이전트는 SandBox 환경을 사용하도록 분리

Phase 5: Dynamic Onboarding & Self-Evolution (✅ 완료)

항목	상태	내용
회사 기반 동적 조직도 생성	✅	사용자의 회사 목표 텍스트 기반 맞춤형 AI 조직 구조 반환
Skill 매뉴얼 자동 생성 및 RAG 연동	✅	프로젝트별 부서 매뉴얼(.md)을 런타임에 자동 생성 및 에이전트 프롬프트에 동적 주입
Self-Evolution (툴 연동 자가발전)	✅	Pi Agent를 통한 `update_skill_manual` 도구 실행으로 LLM 지침서 자체 수정

Phase 6: 고급 도구 및 RAG 파이프라인 연장 (🔷 확장 계획)

항목	상태	내용
모바일 호환성 최적화	⬜	Flutter 웹 접속 시 모바일 반응형 완벽 처리
GitHub 연동 도구	⬜	GitHub API로 저장소 코드 직접 탐색/PR 생성 도구 추가
파일 파싱 고도화	⬜	PDF(사업자등록증 파싱), Excel 문서 기반 LLM 조회 최적화

---

6. 회사 서류 관리

6.1 서류 저장 경로

`E:\geminiclaw\doc\company_docs\`

서류	설명	파일명
AKTA PENDIRIAN	법인설립정관	`akta_pendirian.pdf`
SK KEHAKIMAN	법무부 승인서	`sk_kehakiman.pdf`
NPWP	납세자 번호	`npwp.pdf`
NIB	사업자등록번호 (OSS)	`nib.pdf`
PKKPR	위치적합성 확인서	`pkkpr.pdf`

6.2 에이전트 참조 방법

현재 `.md`, `.txt`, `.json` 파일만 자동 참조.
PDF → 텍스트 변환 필요 (Phase 2에서 PyPDF2 추가 예정).

---

7. 버그 수정 이력

7.1 무한 루프 (2026-02-24 수정)

원인: Reviewer가 `create_agent_node()` 팩토리로 생성 → 항상 `status: "needs_review"` 반환 → graph.py에서 "rejected"로 분기 → 첫 에이전트로 되돌아감 → 무한 루프

증상: `session_20260224_143729_17719186.txt` — 7,652줄의 반복 (legal → reviewer APPROVED → legal → ...)

수정:

1. `agent_factory.py` : Reviewer 텍스트에 "APPROVED" 포함 시 `status: "approved"` 반환
2. `graph.py` : Dispatcher 노드 기반 순차 실행
3. `agent_config.py` : COMPANY_CONTEXT 주입으로 "정보 요청" 루프 방지

---

8. 사용자 매뉴얼 (User Manual)

GeminiClaw AI OS를 효율적으로 사용하기 위한 핵심 가이드입니다.

8.1 시작하기 (Getting Started)

1. **백엔드 서버 실행**: `openclaw_backend` 디렉토리 내 `.env` 파일에 `GEMINI_API_KEY`를 설정합니다. 그 후 터미널에서 `uvicorn main:app --reload --port 8001` 명령어로 서버를 실행합니다.
2. **UI 실행**: `openclaw_app` 디렉토리에서 `flutter run -d web-server --web-hostname 127.0.0.1 --web-port 8080` 명령어로 웹 프론트엔드를 구동합니다.
3. 브라우저에서 `http://127.0.0.1:8080/`에 접속하여 대시보드에 진입합니다.

8.2 새로운 회사/조직 환경 구축 (Onboarding)

1. 좌측 사이드바 하단 아이콘 중 **'Team OS'**(오케스트레이터 뷰) 버튼을 클릭합니다.
2. 우측 상단의 `+ Create Global Project` 버튼을 눌러 새로운 프로젝트를 생성합니다. (예: `AI_Startup_Project`)
3. 바로 아래의 **'Select Project context...'** 드롭다운에서 방금 만든 프로젝트를 선택합니다.
4. **'Target & Goals'** 입력란에 회사의 비전과 현재 목표를 구체적으로 적습니다. (예: "우리는 B2B SaaS 앱을 만드는 3인 개발팀입니다.")
5. `Generate Custom Org & Skills` 버튼을 누릅니다. AI가 목표를 분석하여 최적의 부서를 구성하고 해당 부서들의 매뉴얼 파일을 `openclaw_backend/storage/skills/{project_id}/` 경로에 일괄 생성합니다.

8.3 일상적인 업무 지시 (Tasks & PM)

1. Team OS 화면 하단 **Task Instruction** 입력창에 지시사항을 입력합니다. (예: "다음 주 런칭을 위한 랜딩 페이지 카피라이팅 초안과 개발 계획서 작성해줘")
2. 하단의 **'Deploy Target Team'** 을 누르면 PM Agent가 이를 수신하여, 이전에 구성된 팀원들(부서)에게 서브 태스크를 분할하고 칸반 보드 뷰에 할당합니다.
3. 이후 에이전트들은 자신의 동적 매뉴얼(Skill)을 읽어가며 차례대로 순차 작업을 진행하며, 리뷰어(Reviewer)의 검증을 거친 후 완료됩니다.

8.4 자가 발전 및 매뉴얼 갱신 (Self-Evolution)

- 에이전트들의 작업 결과물 형식이 마음에 들지 않거나, 회사의 규칙이 바뀌었다면 별도로 파일을 수정할 필요가 없습니다.

1. 좌측 사이드바 상단 아이콘 중 **'Pi Assistant'**(채팅 뷰)를 클릭합니다.
2. 하단 채팅창에 다음과 같이 지시합니다:

> *"현재 프로젝트의 개발자(developer) 매뉴얼을 업데이트해 줘. 앞으로 모든 코드 작성 시 주석을 한국어로 달아야 한다는 규칙을 맨 밑에 추가해."*

3. Pi Agent가 내부 `update_skill_manual` 도구를 사용하여 스스로 기존 마크다운 매뉴얼 파일 구조를 해치지 않으면서 규칙을 훌륭하게 추가합니다. 이후의 모든 태스크들은 이 수정된 규

칙을 따르게 됩니다.